

DEPLOYMENT AND GOVERNANCE

Module 7 — Governed AI Feature Delivery

THE PROBLEM WE'RE SOLVING

AI features fail in production when release controls are weak.

-
-
-
-

THE PROBLEM WE'RE SOLVING

AI features fail in production when release controls are weak.

- Prompt and model changes without traceability
-
-
-

THE PROBLEM WE'RE SOLVING

AI features fail in production when release controls are weak.

- Prompt and model changes without traceability
- No rollout guardrails or kill switch
-
-

THE PROBLEM WE'RE SOLVING

AI features fail in production when release controls are weak.

- Prompt and model changes without traceability
- No rollout guardrails or kill switch
- Insufficient logging for audits
-

THE PROBLEM WE'RE SOLVING

AI features fail in production when release controls are weak.

- Prompt and model changes without traceability
- No rollout guardrails or kill switch
- Insufficient logging for audits
- No eval gates in CI/CD

THE PROBLEM WE'RE SOLVING

AI features fail in production when release controls are weak.

- Prompt and model changes without traceability
- No rollout guardrails or kill switch
- Insufficient logging for audits
- No eval gates in CI/CD

Now: define a safe deployment contract for AI features.

WHY THIS MATTERS OPERATIONALLY

-
-
-
-

WHY THIS MATTERS OPERATIONALLY

- Strong prototypes fail without release controls.
-
-
-

WHY THIS MATTERS OPERATIONALLY

- Strong prototypes fail without release controls.
- AI changes happen in prompts, models, and data, not only in code.
-
-

WHY THIS MATTERS OPERATIONALLY

- Strong prototypes fail without release controls.
- AI changes happen in prompts, models, and data, not only in code.
- Every release needs evidence, not confidence alone.
-

WHY THIS MATTERS OPERATIONALLY

- Strong prototypes fail without release controls.
- AI changes happen in prompts, models, and data, not only in code.
- Every release needs evidence, not confidence alone.
- Governance should reduce incident recovery time and change risk.

WHY THIS MATTERS OPERATIONALLY

- Strong prototypes fail without release controls.
- AI changes happen in prompts, models, and data, not only in code.
- Every release needs evidence, not confidence alone.
- Governance should reduce incident recovery time and change risk.

Release discipline is what makes safe iteration possible at scale.

GOVERNANCE-READY RELEASE MODEL

-
-
-
-

GOVERNANCE-READY RELEASE MODEL

- Version prompts, models, and eval sets together
-
-
-

GOVERNANCE-READY RELEASE MODEL

- Version prompts, models, and eval sets together
- Require eval thresholds before deploy
-
-

GOVERNANCE-READY RELEASE MODEL

- Version prompts, models, and eval sets together
- Require eval thresholds before deploy
- Capture traces and decision logs
-

GOVERNANCE-READY RELEASE MODEL

- Version prompts, models, and eval sets together
- Require eval thresholds before deploy
- Capture traces and decision logs
- Provide rollback and fallback playbooks

WHAT "RELEASE-READY" MEANS

-
-
-
-
-

WHAT "RELEASE-READY" MEANS

- Quality thresholds pass on the current golden dataset.
-
-
-
-

WHAT "RELEASE-READY" MEANS

- Quality thresholds pass on the current golden dataset.
- Safety and refusal behaviour validated.
-
-
-

WHAT "RELEASE-READY" MEANS

- Quality thresholds pass on the current golden dataset.
- Safety and refusal behaviour validated.
- Traceability and provenance fields confirmed.
-
-

WHAT "RELEASE-READY" MEANS

- Quality thresholds pass on the current golden dataset.
- Safety and refusal behaviour validated.
- Traceability and provenance fields confirmed.
- Rollback and fallback mechanisms tested.
-

WHAT "RELEASE-READY" MEANS

- Quality thresholds pass on the current golden dataset.
- Safety and refusal behaviour validated.
- Traceability and provenance fields confirmed.
- Rollback and fallback mechanisms tested.
- Ownership and on-call response paths clear.

WHAT "RELEASE-READY" MEANS

- Quality thresholds pass on the current golden dataset.
- Safety and refusal behaviour validated.
- Traceability and provenance fields confirmed.
- Rollback and fallback mechanisms tested.
- Ownership and on-call response paths clear.

Every one of these must be true. A partial checklist is not a release gate.

VERSIONING CONTRACT

Asset	Must be versioned	Why
Prompt assets	Yes	Behaviour changes outside app logic
Model identifier	Yes	Quality, cost, and latency can shift
Eval dataset	Yes	Prevents moving the quality baseline
Policy config	Yes	Compliance behaviour changes

*If an asset changes behaviour, it must have
a version. No exceptions.*

CI/CD AI GATES

-
-
-
-

CI/CD AI GATES

- Block deploy on hard-fail eval criteria.
-
-
-

CI/CD AI GATES

- Block deploy on hard-fail eval criteria.
- Enforce minimum pass rate for key dimensions.
-
-

CI/CD AI GATES

- Block deploy on hard-fail eval criteria.
- Enforce minimum pass rate for key dimensions.
- Check latency and cost against budget thresholds.
-

CI/CD AI GATES

- Block deploy on hard-fail eval criteria.
- Enforce minimum pass rate for key dimensions.
- Check latency and cost against budget thresholds.
- Require trace artefact generation in the pipeline.

CI/CD AI GATES

- Block deploy on hard-fail eval criteria.
- Enforce minimum pass rate for key dimensions.
- Check latency and cost against budget thresholds.
- Require trace artefact generation in the pipeline.

Every gate should have a named owner. Unowned gates get bypassed.

EXAMPLE RELEASE GATE POLICY

- 1.
- 2.
- 3.
- 4.
- 5.

EXAMPLE RELEASE GATE POLICY

1. Schema compliance at or above 99%
- 2.
- 3.
- 4.
- 5.

EXAMPLE RELEASE GATE POLICY

1. Schema compliance at or above 99%
2. Safety and refusal tests: zero hard failures
- 3.
- 4.
- 5.

EXAMPLE RELEASE GATE POLICY

1. Schema compliance at or above 99%
2. Safety and refusal tests: zero hard failures
3. Core task accuracy at or above target threshold
- 4.
- 5.

EXAMPLE RELEASE GATE POLICY

1. Schema compliance at or above 99%
2. Safety and refusal tests: zero hard failures
3. Core task accuracy at or above target threshold
4. P95 latency at or below budget limit
- 5.

EXAMPLE RELEASE GATE POLICY

1. Schema compliance at or above 99%
2. Safety and refusal tests: zero hard failures
3. Core task accuracy at or above target threshold
4. P95 latency at or below budget limit
5. Trace completeness: all required fields present

OBSERVABILITY REQUIREMENTS

-
-
-
-
-

OBSERVABILITY REQUIREMENTS

- Request trace id and correlation across services
-
-
-
-

OBSERVABILITY REQUIREMENTS

- Request trace id and correlation across services
- Prompt, model, and version provenance per run
-
-
-

OBSERVABILITY REQUIREMENTS

- Request trace id and correlation across services
- Prompt, model, and version provenance per run
- Validation outcomes and reason codes
-
-

OBSERVABILITY REQUIREMENTS

- Request trace id and correlation across services
- Prompt, model, and version provenance per run
- Validation outcomes and reason codes
- Tool call intent and result metadata
-

OBSERVABILITY REQUIREMENTS

- Request trace id and correlation across services
- Prompt, model, and version provenance per run
- Validation outcomes and reason codes
- Tool call intent and result metadata
- Fallback and refusal event metrics

OBSERVABILITY REQUIREMENTS

- Request trace id and correlation across services
- Prompt, model, and version provenance per run
- Validation outcomes and reason codes
- Tool call intent and result metadata
- Fallback and refusal event metrics

Observability design starts before release, not after the first incident.

RUNTIME CONTROL PATTERNS

-
-
-
-

RUNTIME CONTROL PATTERNS

- Kill switch for immediate risk containment
-
-
-

RUNTIME CONTROL PATTERNS

- Kill switch for immediate risk containment
- Canary rollout before broad release
-
-

RUNTIME CONTROL PATTERNS

- Kill switch for immediate risk containment
- Canary rollout before broad release
- Feature flags by region or customer tier
-

RUNTIME CONTROL PATTERNS

- Kill switch for immediate risk containment
- Canary rollout before broad release
- Feature flags by region or customer tier
- Fallback path for degraded model behaviour

ROLLBACK VS FALLBACK

Control	Trigger	Effect
Rollback	Bad release behaviour across traffic	Revert to previous version bundle
Fallback	Uncertain or failed single request	Safe deterministic response path

ROLLBACK VS FALLBACK

Control	Trigger	Effect
Rollback	Bad release behaviour across traffic	Revert to previous version bundle
Fallback	Uncertain or failed single request	Safe deterministic response path

Rollback alone does not protect in-flight uncertain requests.
You usually need both.

INCIDENT RESPONSE FLOW

- 1.
- 2.
- 3.
- 4.
- 5.

INCIDENT RESPONSE FLOW

1. Detect regression via alerts or eval monitoring.
- 2.
- 3.
- 4.
- 5.

INCIDENT RESPONSE FLOW

1. Detect regression via alerts or eval monitoring.
2. Triage with trace evidence to locate the failure stage.
- 3.
- 4.
- 5.

INCIDENT RESPONSE FLOW

1. Detect regression via alerts or eval monitoring.
2. Triage with trace evidence to locate the failure stage.
3. Activate kill switch or rollback if traffic is affected.
- 4.
- 5.

INCIDENT RESPONSE FLOW

1. Detect regression via alerts or eval monitoring.
2. Triage with trace evidence to locate the failure stage.
3. Activate kill switch or rollback if traffic is affected.
4. Route affected requests through the fallback path.
- 5.

INCIDENT RESPONSE FLOW

1. Detect regression via alerts or eval monitoring.
2. Triage with trace evidence to locate the failure stage.
3. Activate kill switch or rollback if traffic is affected.
4. Route affected requests through the fallback path.
5. Capture post-incident control improvements as permanent changes.

REGIONAL AND GOVERNANCE CONTEXT

-
-
-
-

REGIONAL AND GOVERNANCE CONTEXT

- Deployment controls may vary by region (EU/US).
-
-
-

REGIONAL AND GOVERNANCE CONTEXT

- Deployment controls may vary by region (EU/US).
- Data residency and audit requirements affect rollout design.
-
-

REGIONAL AND GOVERNANCE CONTEXT

- Deployment controls may vary by region (EU/US).
- Data residency and audit requirements affect rollout design.
- Governance evidence should be available on demand, not assembled after the fact.
-

REGIONAL AND GOVERNANCE CONTEXT

- Deployment controls may vary by region (EU/US).
- Data residency and audit requirements affect rollout design.
- Governance evidence should be available on demand, not assembled after the fact.
- Release decisions should be explainable to non-engineering stakeholders.

COMMON RELEASE ANTI-PATTERNS

-
-
-
-

COMMON RELEASE ANTI-PATTERNS

- Shipping prompt or model changes without rerunning evals.
-
-
-

COMMON RELEASE ANTI-PATTERNS

- Shipping prompt or model changes without rerunning evals.
- No explicit owner for release gate failures.
-
-

COMMON RELEASE ANTI-PATTERNS

- Shipping prompt or model changes without rerunning evals.
- No explicit owner for release gate failures.
- No rehearsed rollback or kill-switch procedure.
-

COMMON RELEASE ANTI-PATTERNS

- Shipping prompt or model changes without rerunning evals.
- No explicit owner for release gate failures.
- No rehearsed rollback or kill-switch procedure.
- Observability added after incidents rather than before release.

MODULE 7 LAB BUILD TARGET

Create a release-readiness pack with:

-
-
-
-
-

MODULE 7 LAB BUILD TARGET

Create a release-readiness pack with:

- Versioned change bundle covering prompt, model, and eval identifiers
-
-
-
-

MODULE 7 LAB BUILD TARGET

Create a release-readiness pack with:

- Versioned change bundle covering prompt, model, and eval identifiers
- Gate criteria with pass/fail outcomes for this release
-
-
-

MODULE 7 LAB BUILD TARGET

Create a release-readiness pack with:

- Versioned change bundle covering prompt, model, and eval identifiers
- Gate criteria with pass/fail outcomes for this release
- Traceability and logging requirements confirmed
-
-

MODULE 7 LAB BUILD TARGET

Create a release-readiness pack with:

- Versioned change bundle covering prompt, model, and eval identifiers
- Gate criteria with pass/fail outcomes for this release
- Traceability and logging requirements confirmed
- Rollback and fallback playbooks with named triggers
-

MODULE 7 LAB BUILD TARGET

Create a release-readiness pack with:

- Versioned change bundle covering prompt, model, and eval identifiers
- Gate criteria with pass/fail outcomes for this release
- Traceability and logging requirements confirmed
- Rollback and fallback playbooks with named triggers
- A written go/no-go recommendation with evidence and owner

MODULE 7 LAB BUILD TARGET

Create a release-readiness pack with:

- Versioned change bundle covering prompt, model, and eval identifiers
- Gate criteria with pass/fail outcomes for this release
- Traceability and logging requirements confirmed
- Rollback and fallback playbooks with named triggers
- A written go/no-go recommendation with evidence and owner

Definition of done: a non-engineer could read this pack and understand whether the feature is safe to release.

SUMMARY

- 1.
- 2.
- 3.
- 4.

SUMMARY

1. **Release-ready AI** requires evidence-backed gates, not confidence alone.
- 2.
- 3.
- 4.

SUMMARY

1. **Release-ready AI** requires evidence-backed gates, not confidence alone.
2. **Version everything** that changes behaviour, not just application code.
- 3.
- 4.

SUMMARY

1. **Release-ready AI** requires evidence-backed gates, not confidence alone.
2. **Version everything** that changes behaviour, not just application code.
3. **Observability before launch** enables safe operation and fast incident recovery.
- 4.

SUMMARY

1. **Release-ready AI** requires evidence-backed gates, not confidence alone.
2. **Version everything** that changes behaviour, not just application code.
3. **Observability before launch** enables safe operation and fast incident recovery.
4. **Rollback and fallback** are both required, and both must be rehearsed.

BRIDGE TO MODULE 8

What we have now:

-

What is next:

-

-

BRIDGE TO MODULE 8

What we have now:

- A complete governed delivery stack: workflow, guardrails, UX, evals, and deployment.

What is next:

-
-

BRIDGE TO MODULE 8

What we have now:

- A complete governed delivery stack: workflow, guardrails, UX, evals, and deployment.

What is next:

- Integrate every pattern into a single working AI feature.
-

BRIDGE TO MODULE 8

What we have now:

- A complete governed delivery stack: workflow, guardrails, UX, evals, and deployment.

What is next:

- Integrate every pattern into a single working AI feature.
- Your first task in Module 8: identify the one gap in your current build that carries the most release risk.

BRIDGE TO MODULE 8

What we have now:

- A complete governed delivery stack: workflow, guardrails, UX, evals, and deployment.

What is next:

- Integrate every pattern into a single working AI feature.
- Your first task in Module 8: identify the one gap in your current build that carries the most release risk.

Module 8 is the final integration and review session. Bring your artefacts.

QUESTIONS?

Module 7 — Governed AI Feature Delivery